

MERN Mastery Program – Week 1 Assignment

Name : S ANAND

Reg No : 99240040808

Course Name : Full Stack Engineering (Organized By INFOZIENT)

Section - A

1. Difference between Traditional Multi-Page Website and Single Page Application (SPA)

Multi-Page Website	Single Page Application (SPA)
1. Entire page reloads for every user action or navigation	1. Page loads once; content updates dynamically without reload
2. Slower due to frequent server requests and full page reloads	2. Faster after initial load due to client-side rendering
3. Less smooth navigation	3. Smooth, app-like user experience
4. Higher server load because each request fetches a new page	4. Reduced server load as only data is exchanged
5. Mainly server-side rendering (HTML, PHP, JSP, etc.)	5. Client-side rendering using JavaScript frameworks like React

2. What problem does React solve in frontend development?

React solves the problem of managing complex and dynamic user interfaces efficiently. It uses a component-based architecture and virtual DOM to update only the necessary parts of the UI, making applications faster, scalable, and easier to maintain.

3. What is a React component? Why are components important?

- A React component is a reusable, independent piece of UI that returns JSX to describe what should appear on the screen.
- Components are important because they **promote reusability, modularity, and maintainability**, allowing large applications to be built from small, manageable pieces.

4. Difference between props and state in React. Who owns each?

PROPS	STATE
1. Props are data passed to a component from its parent	1. State is data managed within a component
2. Props are read-only and cannot be modified by the child	2. State is mutable and can be changed using setState / useState
3. Used to pass data and configuration to components	3. Used to manage dynamic and interactive data
4. Owned by the parent component	4. Owned by the component itself
5. Makes components reusable and configurable	5. Controls component behaviour and re-rendering

5. Why is React declarative rather than imperative?

- React is declarative because developers describe what the UI should look like for a given state, and React automatically updates the DOM.
- This contrasts with imperative programming, where developers must manually control how the UI updates step by step.

SECTION – B

6. Write a simple React functional component that displays a name passed as a prop. Explain how props are received inside the component?

Props (short for properties) are read-only inputs passed from a parent component to a child component in React. They are used to send data and configuration to components, making them reusable and dynamic.

Code Example:

```
function Welcome(props) {  
  return <h1>Welcome, {props.name}</h1>;  
}  
export default Welcome;
```

Usage of the Component (Parent Component)

```
function App() {  
  return <Welcome name="Anand" />;  
}
```

Output: Welcome, Anand

Explanation:

In React, props (properties) are used to pass data from a parent component to a child component.

- In the parent component (App), the value name="Anand" is passed to the Welcome component as a prop.
- In the child component, props are received as a parameter in the function (props).
- The value of the prop is accessed using props.name.
- The component then displays the passed value dynamically in the UI.

7. What happens if useState is updated twice with the same value in React? Explain with example.

Case 1: Using Normal State Update

```
import { useState } from "react";  
  
function CounterWrong() {  
  const [count, setCount] = useState(0);  
  const handleClick = () => {  
    setCount(count + 1);  
    setCount(count + 1);  
  };  
  return (  
    <div>  
      <h2>Wrong Way Counter</h2>  
      <p>Count: {count}</p>  
      <button  
onClick={handleClick}>Increase</button>  
    </div>  
  );  
}  
  
export default CounterWrong;
```

Result: Count increases by **1** only.

Step-by-Step Explanation

1. When the button is clicked, React starts executing the function.
2. count has an initial value (for example, 0).
3. The first setCount(count + 1) schedules an update to change the value from 0 to 1.
4. React does not update the state immediately.

5. The second `setCount(count + 1)` still uses the old value (0), not the updated one.
6. React batches both updates together for performance.
7. Since both updates calculate the same result (1), the final state becomes 1.
8. The component re-renders once with the updated value.

Case 2: Using Functional State Update

```
import { useState } from "react";

function CounterCorrect() {
  const [count, setCount] = useState(0);
  const handleClick = () => {
    setCount(prevCount => prevCount + 1);
    setCount(prevCount => prevCount + 1);
  };
  return (
    <div>
      <h2>Correct Way Counter</h2>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Increase</button>
    </div>
  );
}

export default CounterCorrect;
```

Result: Count increases by 2.

Step-by-Step Explanation

1. React executes the first update and passes the current state (0) as `prevCount`.
2. The state updates to 1.
3. React executes the second update and passes the **latest value (1)** as `prevCount`.

4. The state updates to 2.
5. React completes batching and re-renders the component.

8. Explain the execution order of a React component when state changes. Mention rendering and effect execution in your explanation.

1. State Update is Triggered

A state update occurs when `setState()` (class components) or `setState` from `useState()` (functional components) is called.

Example:

```
setCount(count + 1);
```

At this point, the state is not updated immediately.

2. React Batches State Updates

React groups (batches) multiple state updates together for better performance. This avoids unnecessary re-renders and improves application efficiency.

3. Component Re-render is Scheduled

React schedules a re-render of the component whose state has changed. Re-render means **re-executing the component function**, not reloading the page.

4. Component Function Executes Again

- The entire component function runs again using the **updated state** and **current props** to produce **new JSX**.
- This JSX represents a **new Virtual DOM tree**.

5. Virtual DOM Comparison (Reconciliation)

React compares:

- Previous Virtual DOM
- New Virtual DOM

This process is called **reconciliation**.
React identifies **only the elements that have changed**.

6. Real DOM Update (Commit Phase)

- React updates **only the changed parts** of the actual browser DOM instead of rewriting the entire page.
- This makes React fast and efficient.

7. Browser Paints Updated UI

The browser renders the updated DOM on the screen.

8. useEffect() Hook Executes

After the DOM update:

- `useEffect()` runs if its dependency array conditions are met.
- Side effects like API calls, timers, or subscriptions are executed here

FLOWCHART:

